SOMAIYA VIDYAVIHAR

**K J Somaiya Institute of Technology**
**An Autonomous Institute Permanently Affiliated to the University of Mumbai**

**Heer Haresh Panchal (BS4 (64))**

# EXPERIMENT NO. 02

## Aim:

Implementation of Queue using Array for real-world application.

## Objective:

- To implement a Linear Queue using an array.
- To perform Insert, Delete, and Display operations.
- To understand FIFO (First In First Out) behavior.
- To handle Overflow and Underflow conditions.

**Software Used:** DOSBox, Turbo C ++

## Theory:

A Queue is a linear data structure in which insertion and deletion of elements are performed at specific ends. It follows the FIFO (First In, First Out) principle, which means the element inserted first into the queue is the first element to be removed. A common real-world example of a queue is a line of people waiting at a ticket counter, where the person who arrives first is served first.

A Linear Queue can be implemented using an array. In this implementation, two variables, front and rear, are used to keep track of the elements in the queue. The front points to the first element available for deletion, while the rear points to the last element inserted into the queue.

For the real-world application, the queue is used for heatwave alert prediction based on temperature:

- Temperature >= 45°C: Red Alert (Enqueue Operation)

- Temperature >= 43°C: Orange Alert (Enqueue Operation)
- Temperature >= 40°C: Yellow Alert (Enqueue Operation)
- Temperature < 40°C: All active alerts are removed using Dequeue operations.

The program performs Enqueue, Dequeue, and Display operations using a menu-driven approach. It also handles Overflow when the queue is full and Underflow when the queue is empty.

## Program 2.1 :

```c
#include <stdio.h>

#define MAX 100

int queue[MAX];
int front = -1, rear = -1;
int queueSize;

/* Insert temperature into queue */
void enqueue(int temp)
{
    if (rear == queueSize - 1)
    {
        printf("Queue Overflow!\n");
        return;
    }

    if (front == -1)
        front = 0;

    rear++;
    queue[rear] = temp;

    if (temp >= 45)
        printf("Red Alert!\n");
    else if (temp >= 43)
        printf("Orange Alert!\n");
    else if (temp >= 40)
        printf("Yellow Alert!\n");
}

/* Remove temperature from queue */
void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("Queue Underflow!\n");
        return;
    }

    printf("Removed temperature reading: %d\n", queue[front]);
    front++;

    if (front > rear)
    {
        front = rear = -1;
    }
}

/* Display all active readings */
void display()
{
    int i;

    if (front == -1 || front > rear)
    {
        printf("No active alerts in queue.\n");
        return;
    }

    printf("Active alerts: ");

    for (i = front; i <= rear; i++)
    {
```

```
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
    int choice, temperature;

    printf("Enter the size of Queue (Maximum Size = 100): ");
    scanf("%d", &queueSize);

    if (queueSize <= 0 || queueSize > MAX)
    {
        printf("Invalid queue size!\n");
        return 0;
    }

    do
    {
```

```
        printf("\n--- Heatwave Alert Queue ---\n");
        printf("1. Enqueue Temperature Reading\n");
        printf("2. Dequeue Active Alert\n");
        printf("3. Display Active Alerts\n");
        printf("4. Exit\n");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter temperature reading: ");
                scanf("%d", &temperature);

                if (temperature < 40)
                {
                    printf("Temperature is below 40°C.\n");
                    printf("No heatwave alert. Clearing active alerts...\n");

                    while (front != -1 && front <= rear)
                        dequeue();
                    }
                }
                else
                {
                    enqueue(temperature);
                }
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Program ended.\n");
                break;
```

```
            default:
                printf("Invalid choice!\n");
        }

    } while (choice != 4);

    return 0;
}
```

## Output:

```
Enter the size of Queue (Maximum Size = 100): 3

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 1
Enter temperature reading: 40
Yellow Alert!

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 1
Enter temperature reading: 44
Orange Alert!

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 1
Enter temperature reading: 50
Red Alert!

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit
```

```
Enter your choice: 3
Active alerts: 40 44 50

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 2
Removed temperature reading: 40

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 3
Active alerts: 44 50

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 1
Enter temperature reading: 49
Queue Overflow!

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 2
Removed temperature reading: 44
```

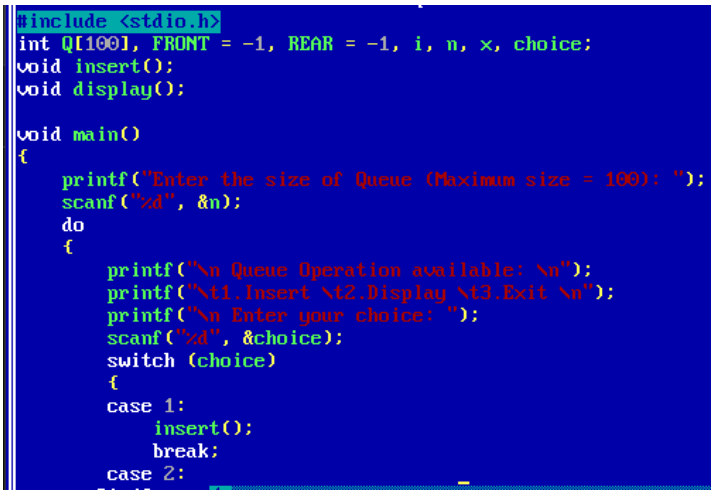
```
--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

Enter your choice: 2
Queue Underflow!

--- Heatwave Alert Queue ---
1. Enqueue Temperature Reading
2. Dequeue Active Alert
3. Display Active Alerts
4. Exit

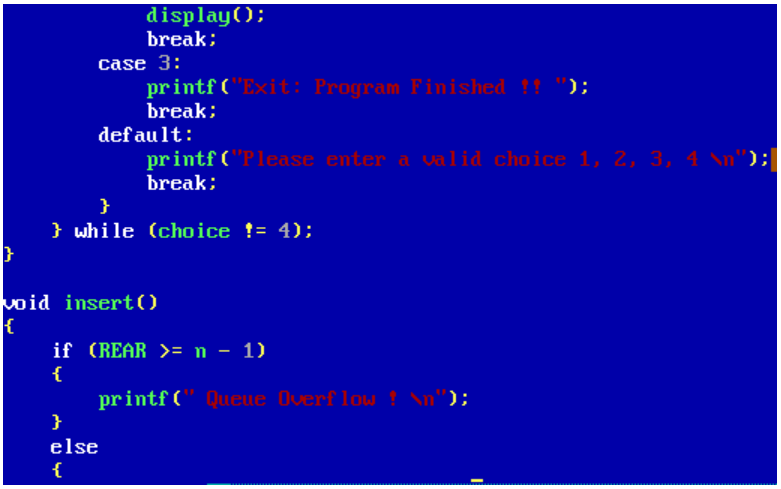
Enter your choice: 1
Enter temperature reading: 35
Temperature is below 40C.
No heatwave alert. Clearing active alerts...
```

## Program 2.2 :



```c
#include <stdio.h>
int Q[100], FRONT = -1, REAR = -1, i, n, x, choice;
void insert();
void display();

void main()
{
    printf("Enter the size of Queue (Maximum size = 100): ");
    scanf("%d", &n);
    do
    {
        printf("\n Queue Operation available: \n");
        printf("\t1.Insert \t2.Display \t3.Exit \n");
        printf("\n Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
        case 1:
            insert();
            break;
        case 2:
```



```c
            display();
            break;
        case 3:
            printf("Exit: Program Finished !! ");
            break;
        default:
            printf("Please enter a valid choice 1, 2, 3, 4 \n");
            break;
        }
    } while (choice != 4);
}

void insert()
{
    if (REAR >= n - 1)
    {
        printf(" Queue Overflow ! \n");
    }
    else
    {
```

```
        printf(" Enter the element to insert: ");
        scanf("%d", &x);
        REAR++;
        Q[REAR] = x;
        if (FRONT == -1)
        {
            FRONT = 0;
        }
    }
}

void display()
{
    if (REAR < 0)
    {
        printf(" Queue is empty ! \n");
    }
    else
    {
        printf(" The elements in the Queue are: \n");
        for (i = FRONT; i < n; i++)
```

```
        {
            printf(" %d ", Q[i]);
        }
        printf("\n");
    }
}
```

## Output :

```
Enter the size of Queue (Maximum size = 100): 3

 Queue Operation available:
       1.Insert        2.Display       3.Exit

 Enter your choice: 1
 Enter the element to insert: 10

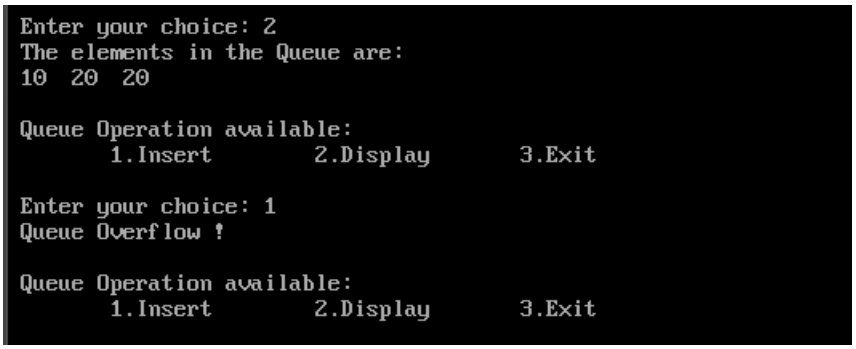
 Queue Operation available:
       1.Insert        2.Display       3.Exit

 Enter your choice: 1
 Enter the element to insert: 20

 Queue Operation available:
       1.Insert        2.Display       3.Exit

 Enter your choice: 1
 Enter the element to insert: 20

 Queue Operation available:
       1.Insert        2.Display       3.Exit
```




## Outcome:

The Linear Queue was successfully implemented using an array. Insert, Delete, and Display operations were performed according to the FIFO principle.

## Conclusion:

The program demonstrates the implementation and basic operations of a Linear Queue using an array, including handling of Overflow and Underflow conditions.